

Agnic answers whether an agent can complete a checkout.

We answer what the fan approved — and whether they paid exactly that.

TEAM

Team Loong

PRESENTED BY

Eason · online

BUILDS ON

Agnic HTTP API · Stripe

The merchant's final total is not knowable in advance.

The shop adds tax at checkout. So “the fan approves a total” cannot be implemented literally — and the obvious workaround is wrong twice.

The obvious implementation

- Authorise a ceiling, capture the whole thing
- Refund the difference afterwards
- Two statement lines for one gift
- A refund fee on every order
- A window where the fan's money is simply gone

What we do instead

- Fan approves a **ceiling**, and the UI says so
- We hold the money — we don't take it
- The shop's real total comes back from the settlement
- We capture **once**, for the number we showed
- The difference is released, not refunded

A ceiling is never presented as a total. When the shop charges less, the saving **goes back to the fan** — we do not keep the difference.

Two rails, deliberately asymmetrical.

WE BUILD

Creator accounts · wishlist curation · the fan's payment rail on Stripe · the approval record · the order state machine · operator tooling for the cases that need a human.

AGNIC HANDLES

Finding products · quoting delivery to a protected address · executing the merchant checkout · the approval handshake when a shop demands one.

list Creator adds a product by link	approve Fan approves a ceiling	hold Stripe authorises, no capture	dispatch Agnic buys from the shop	capture Exactly the number shown
--	-----------------------------------	---------------------------------------	--------------------------------------	-------------------------------------

Every state change passes one machine — eleven states, and an illegal transition **throws** rather than being silently recorded. There is no path to a merchant purchase that skips the fan's explicit approval: `src/orders/dispatcher.ts` is the only code in the entire build that can spend money.

We captured the ceiling and kept \$4.94 having shown \$2.99.

FAN APPROVED
(THE CEILING)

17.94

14.95 estimate + 2.99 fee

MERCHANT CHARGED
(REAL SETTLEMENT)

13.00

Agnic, the real shop

CAPTURED
AT THE TIME

17.94

the whole ceiling – the bug

CAPTURED
TODAY'S CODE

15.99

13.00 + the fee we showed

BEFORE

Fee shown to the fan **2.99**

Fee actually kept **4.94**

Overcharged, silently **1.95**

AFTER

Fee shown to the fan **2.99**

Fee kept **2.99**

Released back instead **1.95**

Found against the **live sandbox**. The shop charged **less** than the ceiling, and no test double can produce that — which is exactly why a fully green suite never caught it. The refusal to capture more than was shown is now a function, not a convention.

The creator's address never reaches the fan.

```
CreatorAddress.ciphertext
v1:9fKq2R ... A7mZ0t ... x81LpQ==

// AES-256-GCM, encrypted before the row is written.
// A missing key throws. There is no plaintext fallback.
```

The digest beside it is deliberately **not** encrypted — it is taken over the **plaintext**. If it moved with the encryption key, rotating that key would silently cancel every outstanding approval. The order carries the digest, never the address.

FAN PAYLOAD

no address field exists to leak – not redacted, absent

OPERATOR PAYLOAD

```
... refused: postal_address.street_address, address_locality,
postal_code
```

The guard names the offending fields and refuses to build the payload — and there is a test that fails if it ever becomes vacuous. When the creator moves house, the outstanding order **refuses** rather than shipping to the wrong door.

What is real, and what is deliberately scoped.

REAL, END TO END

- A real merchant settlement: the shop was charged **13.00** through Agnic — `af_ord_mu8yhh4hhi7zo6az`
- A real Stripe PaymentIntent in test mode, authorised then captured — `pi_3UHWgs...1Z59KStW`
- Real address encryption at rest, AES-256-GCM, with no plaintext path
- Real operator authentication, real refund route, real audit trail

SCOPED, AND STATED AS SUCH

- The exhibit order's fan rail ran on the in-memory double — the `pay_...` refs show it. That is what lets us *show* the bug without moving money
- Stripe is in test mode: the amounts are real, the funds are not
- One sandbox merchant's gateway accepts the test card; its storefront has no public product URLs, so its catalogue is synced by script

202	21 / 21	11	2
TESTS PASSING	PREFLIGHT CHECKS	ORDER STATES	LIVE SETTLEMENTS